

Advanced SQL Development Project

Mini Store Manager

Project Report

Student: HARINESH MUTHU KUMAR

Course: Advanced SQL

Academic Year: 2025–2026

1. Project Overview

1.1 Introduction

The Mini Store Manager is a web based store management application developed using Python, Flask and SQLite. The purpose of the project is to provide a simple interface for managing products, customers and customer orders while demonstrating advanced SQL development techniques.

The application provides five main areas: Dashboard, Products, Customers, Orders and Analytics. SQLite is used as the database management system and Flask is used as the Python web framework.

1.2 Technologies Used

- Python 3.13.5
- Flask
- SQLite
- HTML/CSS
- SQL

SQLite is accessed through Python's built-in sqlite3 module.

1.3 Project Structure

```
mini-store-manager/  
├── database/  
│   ├── schema.sql  
│   ├── seed.sql  
│   └── store.db  
├── src/  
│   ├── app.py  
│   ├── database.py  
│   ├── routes/  
│   └── templates/  
├── report/  
│   └── report.pdf  
├── .gitignore  
├── README.md  
└── requirements.txt
```

2. Database Design

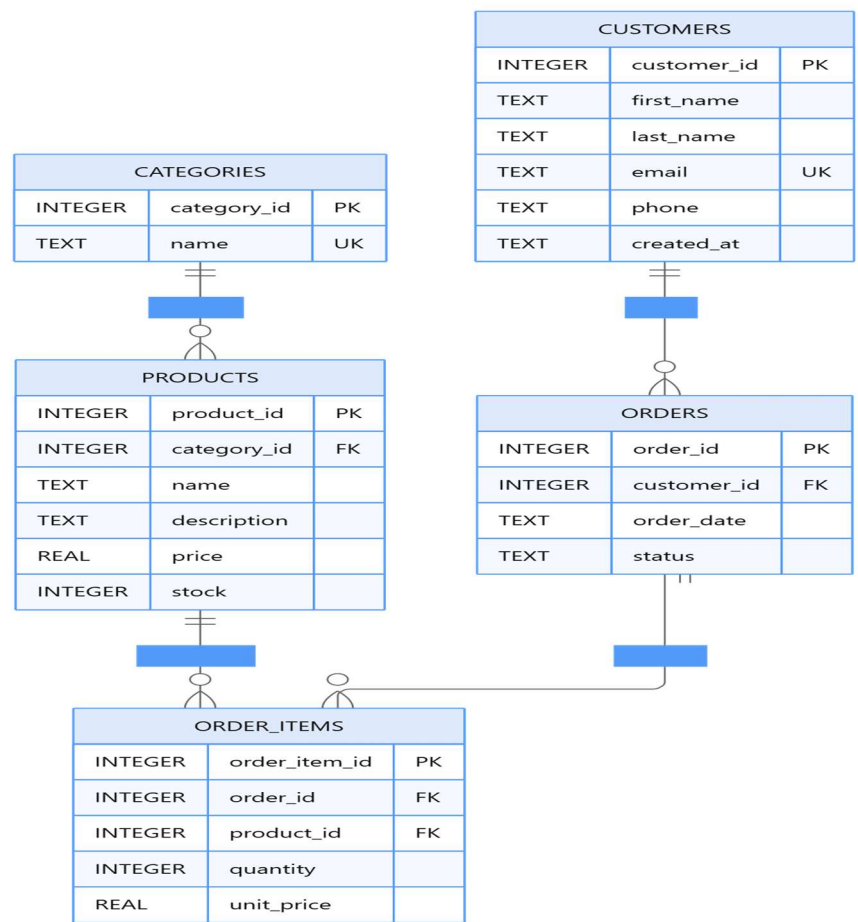
2.1 Database Tables

The database consists of five main tables:

Table	Purpose
categories	Stores product categories
products	Stores products, prices and stock
customers	Stores customer information
orders	Stores customer orders
order_items	Stores the products and quantities belonging to each order

The order_items table connects orders and products and records the quantity and unit price of each product sold.

2.2 ER Diagram



2.3 Database Constraints

The database uses primary keys to uniquely identify records and foreign keys to maintain relationships between related tables. It also uses NOT NULL, UNIQUE and CHECK constraints and default values.

Product prices and stock quantities cannot be negative. Customer email addresses are unique. Order item quantities must be greater than zero. These constraints help maintain data integrity.

2.4 Seed Data

The final database contains the required minimum seed data:

Data	Records
Categories	5
Products	20
Customers	12
Orders	25
Order Items	50

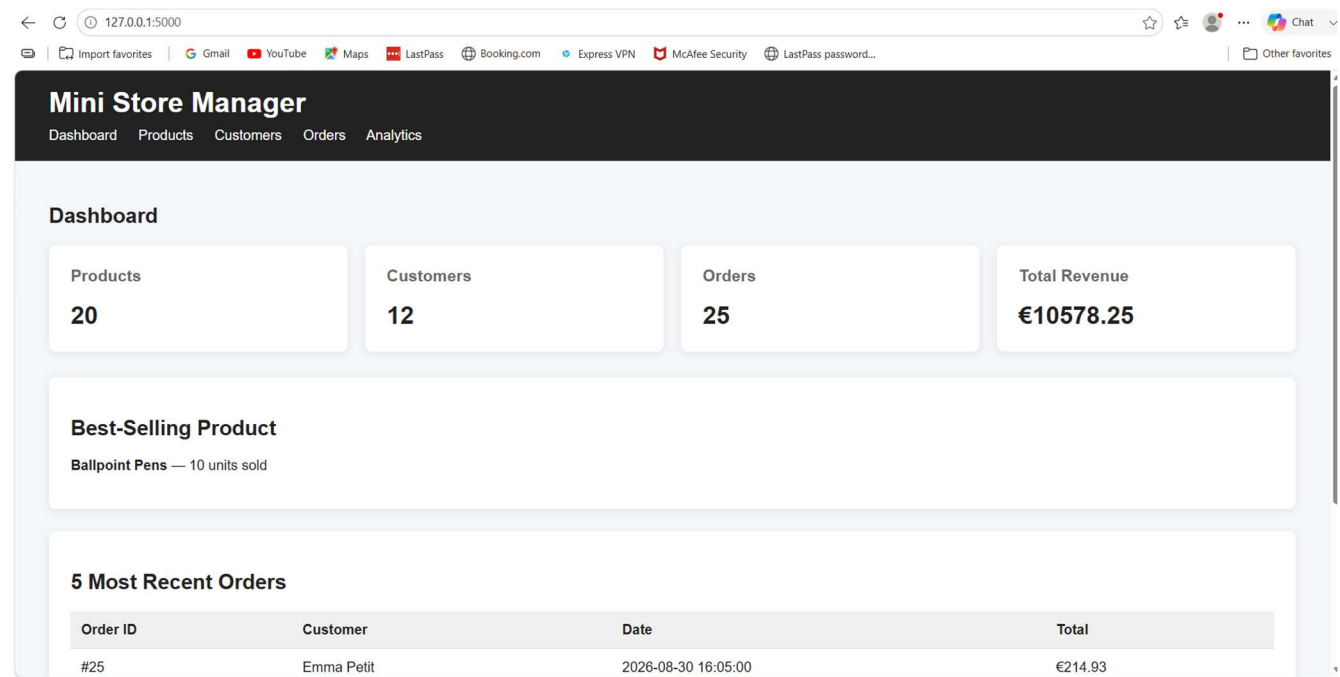
Database validation confirmed that all foreign-key relationships are valid. The seeded data also includes Screen Protector as a product that has never been ordered.

3. Application Features

3.1 Dashboard

The Dashboard provides an overview of the store, including the number of products, customers and orders, total revenue, the best-selling product and the five most recent orders.

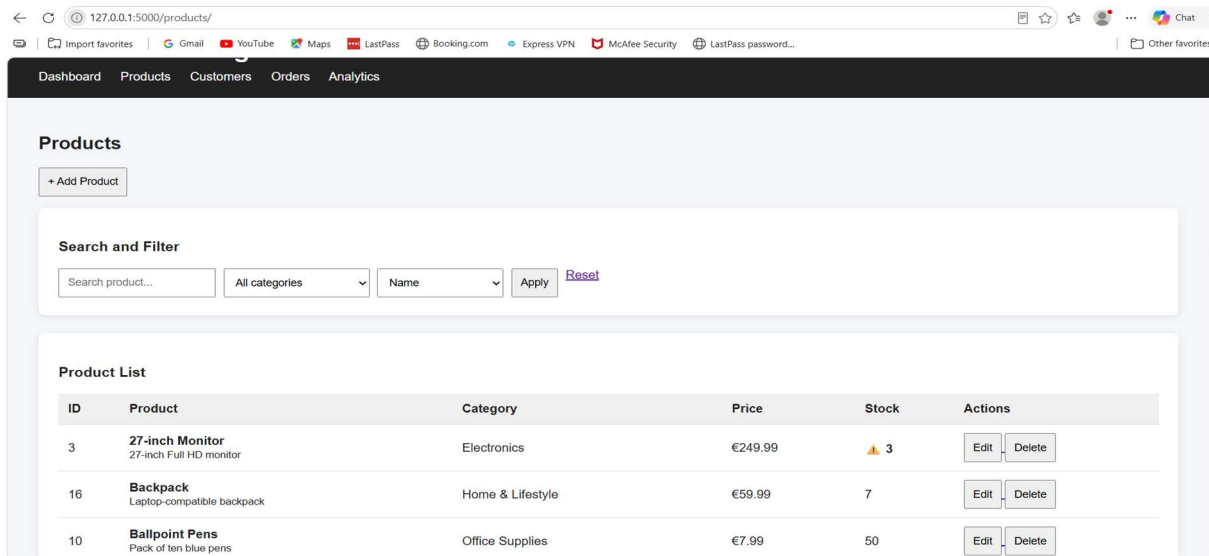
In the tested database, the dashboard displayed 20 products, 12 customers and 25 orders. Ballpoint Pens was displayed as the best-selling product with 10 units sold.



3.2 Product Management

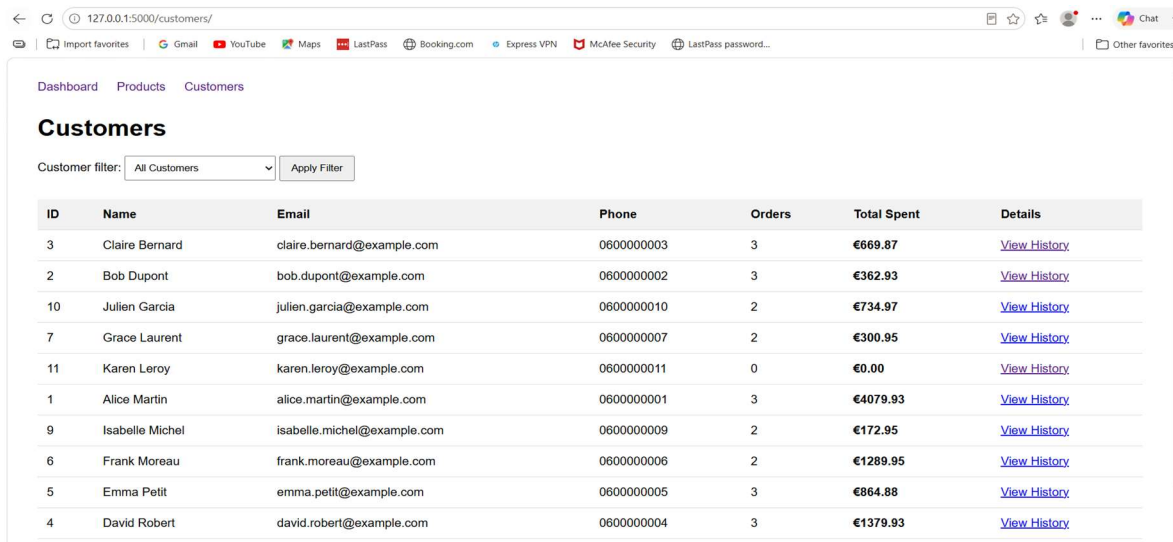
The Products page provides CRUD functionality. Users can view, add, edit and delete products, as well as search, filter by category and sort the product list. Low-stock products are visually identified.

Products that have already been used in an order cannot be deleted. This protects historical order relationships.



3.3 Customer Management

The Customers page supports filtering customers by whether they have orders. The customer detail page displays order history and total spending. This demonstrates the relationship between customers and orders and the use of aggregate SQL queries.



3.4 Order Management

The Orders section allows users to create and view orders. During order creation, the application validates the customer, products and requested quantities, checks available stock, inserts the order and order items, updates stock and commits the transaction.

If an operation fails, the transaction is rolled back. This prevents incomplete orders and protects stock consistency.

127.0.0.1:5000/orders/

Dashboard Products Customers Orders

Orders

+ Create Order

Order ID	Customer	Order Date	Status	Total	Details
25	Emma Petit	2026-08-30 16:05:00	Completed	€214.93	View
24	David Robert	2026-08-22 12:35:00	Completed	€239.98	View
23	Claire Bernard	2026-08-12 15:15:00	Completed	€549.97	View
22	Bob Dupont	2026-08-01 09:20:00	Completed	€89.98	View
21	Alice Martin	2026-07-20 11:45:00	Completed	€1329.98	View
20	Julien Garcia	2026-07-11 14:05:00	Completed	€689.98	View
19	Isabelle Michel	2026-07-02 10:55:00	Completed	€72.97	View
18	Hugo Simon	2026-06-23 16:40:00	Completed	€91.91	View

3.5 Analytics

The Analytics page implements the six required analyses: A) revenue by category; B) top 5 customers; C) products that have never been ordered; D) average order value and orders above average; E) low-stock products; and F) sales by month.

127.0.0.1:5000/analytics/

Analytics

A. Revenue by Category

Category	Revenue
Electronics	€8099.89
Computer Accessories	€939.82
Home & Lifestyle	€909.88
Mobile Accessories	€392.88
Office Supplies	€235.78

B. Top 5 Customers by Total Spending

Customer	Email	Total Spent
Alice Martin	alice.martin@example.com	€4079.93
David Robert	david.robert@example.com	€1379.93
Frank Moreau	frank.moreau@example.com	€1289.95
Emma Petit	emma.petit@example.com	€864.88
Julien Garcia	julien.garcia@example.com	€734.97

4. Advanced SQL and Analytics

4.1 Aggregation and GROUP BY

The application uses COUNT(), SUM() and AVG() for record counts, revenue, order totals and average order value. GROUP BY is used to produce grouped analytical results such as revenue by product category.

4.2 Joins

INNER JOIN and LEFT JOIN are used to combine related information from categories, products, customers, orders and order_items. LEFT JOIN is particularly useful where records must remain visible even when no related record exists.

4.3 NOT EXISTS

The never-ordered analysis uses NOT EXISTS to identify products that do not appear in any order item:

```
WHERE NOT EXISTS (  
    SELECT 1  
    FROM order_items oi  
    WHERE oi.product_id = p.product_id  
)
```

4.4 Subqueries and CTEs

The average order analysis uses a derived subquery to calculate order totals and a Common Table Expression (CTE) to compare individual order values against the overall average.

4.5 Parameterized Queries

User input is passed to SQL queries using parameters rather than being concatenated into SQL statements. This approach improves safety and helps prevent SQL injection.

```
connection.execute(  
    "SELECT * FROM products WHERE name LIKE ?",  
    (search_pattern,)  
)
```

4.6 Transactions

Order creation uses a database transaction. The order, order items and stock updates are performed as one logical operation. If any part fails, the transaction is rolled back, helping maintain consistent data.

5. Testing and Validation

5.1 Database Record Validation

The final database was checked using a Python verification script. The results were:

```
categories : 5  
products   : 20  
customers  : 12  
orders     : 25  
order_items : 50
```

5.2 Foreign-Key Validation

Foreign-key relationships were tested and the final result was:

Foreign key violations:
None - all foreign keys are valid.

5.3 Never-Ordered Product Test

A SQL query using NOT EXISTS was executed to verify the never-ordered requirement. The result identified product 20, Screen Protector, as not appearing in order_items.

5.4 Functional Testing

- Dashboard loaded correctly.
- Product search, category filtering and sorting worked.
- Product CRUD functionality was tested.
- Customer filters and customer history worked.
- Order creation and order details worked.
- Insufficient-stock validation prevented an invalid order.
- Analytics sections A–F loaded correctly.

6. Conclusion

The Mini Store Manager successfully implements a Flask web application connected to an SQLite database. The database contains the required five tables and the required seed data, with integrity maintained through primary keys, foreign keys and validation constraints.

The application provides the required Dashboard, Products, Customers, Orders and Analytics functionality. It also demonstrates advanced SQL concepts including joins, aggregation, grouping, subqueries, Common Table Expressions, NOT EXISTS, parameterized queries and transactions.

Testing confirmed that the database contains the required records, foreign-key relationships are valid, and the main application functions operate correctly. The final project is prepared for submission with source code, SQL schema, seed data, SQLite database, README and final report.